QueryAPIGate HOW-TO GUIDE 53 · GOVERN

Send the audit log to your SIEM, and prove nobody edited it

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=53-send-the-audit-log-to-your-siem

Time: 20 minutes. **You'll end up with:** every administrative change signed into a hash chain, streamed over mutual TLS to your SIEM as it happens - and a check, in one command, that what QueryAPIGate holds, and what your SIEM received, is what was written.

> **Stable** - the SIEM stream is under the compatibility promise since 0.32 (what that means), as the chain itself already was.

Every command and output below was run against a real server, with **rsyslog 8.2604** as the SIEM - RFC 5425 syslog over TLS, requiring a client certificate - and an **OpenTelemetry Collector 0.160.0** taking OTLP logs beside it: the receivers `.github/scripts/siem_receivers.sh` starts, the same ones CI tests the stream against on every push.

Step 1: Sign the chain

```shell
python -c "import secrets; print(secrets.token_hex(32))"   # the audit key
QUERYAPIGATE_AUDIT_KEY=<that key>
QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE=/var/log/queryapigate/audit.jsonl   # a copy on disk too
```

Keep the key outside the store, as you keep `QUERYAPIGATE_SECRET_KEY`. Every audit entry is then signed with it (`hmac-sha256:...`), chained to the one before - someone who can write to the store but hasn't the key can't re-sign an edit. Setting it is the chain's first entry here: `audit_key_set`.

Step 2: The SIEM side

rsyslog, taking TLS on 6514 and accepting only clients with a certificate from your CA:

```text
global(DefaultNetstreamDriver="ossl" DefaultNetstreamDriverCAFile="/certs/ca.pem"
       DefaultNetstreamDriverCertFile="/certs/server.pem"
DefaultNetstreamDriverKeyFile="/certs/server.key")
module(load="imtcp" StreamDriver.Name="ossl" StreamDriver.Mode="1"
StreamDriver.AuthMode="x509/certvalid")
input(type="imtcp" port="6514")
template(name="parsed" type="list") {
  property(name="app-name") constant(value="\t") property(name="msgid") constant(value="\t")
  property(name="structured-data") constant(value="\t") property(name="msg") constant(value="\n")
}
action(type="omfile" file="/out/rsyslog.log" template="parsed")
```

syslog-ng takes the same messages - `syslog(transport("tls") ... tls(... peer-verify(required-trusted)))`, the parser Splunk Connect for Syslog is built on - and so do Splunk's syslog input and Sentinel's agent. For OTLP, a Collector with an `otlp` receiver over HTTP.

Step 3: Point QueryAPIGate at it

```shell
QUERYAPIGATE_AUDIT_SYSLOG_URL=tls://siem.example.com:6514
QUERYAPIGATE_AUDIT_SYSLOG_CA=/etc/queryapigate/siem-ca.pem   # if the SIEM's certificate isn't from a public CA
QUERYAPIGATE_AUDIT_OTLP=1                                    # and OTLP logs, acknowledged per batch
OTEL_EXPORTER_OTLP_LOGS_ENDPOINT=http://otel-collector:4318/v1/logs
```

Started without a client certificate, rsyslog refuses the connection - and that shows straight away, with nothing counted as delivered, while OTLP goes on:

```json
GET /api/v1/audit/delivery
{"output": "syslog", "delivered_seq": 0, "head_seq": 1, "behind": 1,
 "last_error": "SSLError: [SSL: TLSV13_ALERT_CERTIFICATE_REQUIRED] tlsv13 alert certificate required (_ssl.c:2570)"}
{"output": "otlp", "delivered_seq": 1, "head_seq": 1, "behind": 0, "last_error": null}
```

A certificate from a CA rsyslog doesn't trust is refused the same way (`tlsv1 alert unknown ca`). Add yours:

```shell
QUERYAPIGATE_AUDIT_SYSLOG_CERT=/etc/queryapigate/client.pem
QUERYAPIGATE_AUDIT_SYSLOG_KEY=/etc/queryapigate/client.key
```

```json
{"output": "syslog", "delivered_seq": 1, "head_seq": 1, "behind": 0, "last_error": null}
```

Step 4: Changes, as the SIEM sees them

A key created, its rate limit changed, its secret rotated - and later deleted. rsyslog's file, as rsyslog parsed each message - APP-NAME, MSGID, structured data, the message:

```text
queryapigate   audit_key_set   [meta sequenceId="1"]  {"action": "audit_key_set", "actor": "startup", "changes": {"entries...
queryapigate   create_key      [meta sequenceId="2"]  {"action": "create_key", "actor": "admin", "changes": {"active": true, ...
queryapigate   update_key      [meta sequenceId="3"]  {"action": "update_key", "actor": "admin", "changes": {"rate_limit": {"...
queryapigate   rotate_api_key  [meta sequenceId="4"]  {"action": "rotate_api_key", "actor": "admin", "changes": {"grace_m...
queryapigate   delete_key      [meta sequenceId="5"]  {"action": "delete_key", "actor": "admin", "changes": {"active": true, ...
```

RFC 5424, facility 13 (log audit): the action is the message ID, `seq` is `[meta sequenceId]`, and the message is the entry with its `seq`, `prev_hash` and `hash` - what the SIEM indexes, and what it drops a repeat by. The Collector got each as a log record - `service.name` `queryapigate`, scope `queryapigate.audit` - with the entry as its body and these attributes, here for entry 3:

```json
{"queryapigate.audit.seq": "3",
 "queryapigate.audit.hash": "hmac-
sha256:9278fc6bbe80696c8f0ad6590e4aab03c7d017b207110f707878ee09c23ca466",
 "queryapigate.audit.prev_hash": "hmac-
sha256:b318f06633e30e89d3294f920aae2f3fa6aaeaaa47b3c1733dae76face28371d",
 "queryapigate.audit.action": "update_key", "queryapigate.audit.actor": "admin",
 "queryapigate.audit.target": "partner-acme"}
```

Step 5: When the SIEM is down

The key was deleted while rsyslog was stopped:

```json
{"output": "syslog", "delivered_seq": 4, "head_seq": 5, "behind": 1,
 "last_error": "ConnectionRefusedError: [Errno 111] Connection refused"}
{"output": "otlp", "delivered_seq": 5, "head_seq": 5, "behind": 0, "last_error": null}
```

Nothing is lost: each output's cursor is in the store, and entries it hasn't had are kept even past `QUERYAPIGATE_AUDIT_LOG_LIMIT` (up to ten times it). Each output is on its own - OTLP kept up throughout. 15 seconds after rsyslog started again, it had caught up:

```json
{"output": "syslog", "delivered_seq": 5, "head_seq": 5, "behind": 0, "last_error": null}
```

More than five minutes behind raises `audit_delivery_behind`. With several instances, one delivers at a time - the one holding the output's lease - and another takes over if it stops.

Step 6: Prove it - from the store, and from the SIEM

```shell
queryapigate audit verify
# Intact: entries 1 to 5, keyed with QUERYAPIGATE_AUDIT_KEY.
queryapigate audit verify --file /var/log/queryapigate/audit.jsonl
# Intact: entries 1 to 5, keyed with QUERYAPIGATE_AUDIT_KEY.
queryapigate audit verify --file rsyslog.log          # the SIEM's own copy, as rsyslog wrote it
# Intact: entries 1 to 5, keyed with QUERYAPIGATE_AUDIT_KEY.
```

`--file` reads what a SIEM wrote as it is: syslog lines, the entry the JSON at the end of each. Delivery is at least once, so a SIEM's copy may hold an entry twice, or out of order after a retry - an exact repeat (the same `seq` and `hash`) is dropped and counted, and entries are checked in `seq` order. A repeat that differs is reported (`conflicting copies`, naming both lines): one of the two isn't what was written.

The SIEM's copy with one entry changed - the rate limit made `6000/minute`:

```text
BROKEN at entry 3 (update_key, 2026-10-13 13:22:55): changed
```

(exit 1). The store is held to the same: the server checks it every five minutes and raises `audit_chain_broken` at the first entry that doesn't verify - changed, removed, inserted, reordered, or re-written without the key.

The SIEM's copy is what catches what the chain can't on its own - the newest entries deleted from the store, or a rewrite by someone holding both the store and the key: check the two against each other, and the SIEM's against itself, with the same command. Without the key, nothing can be checked, and the command says so rather than claim it's intact:

```text
Not checked: all 5 entries are keyed, and QUERYAPIGATE_AUDIT_KEY is not set - set it to the key they were written with.
```

(exit 2).

Good to know

- **Rotating the audit key:** `QUERYAPIGATE_AUDIT_KEY=<new>,<old>` - the first signs, both verify; startup records `audit_key_rotated`. Keep the old key while entries signed with it are kept.
- **Syslog over TCP has no acknowledgement.** A SIEM that closes its end, or refuses the certificate, is noticed before anything is written into the connection - stopping rsyslog with a connection open is noticed, CI checks. One that dies without closing it can lose the batch sent into it. If that matters, use OTLP logs, acknowledged per batch - or a forwarder next to QueryAPIGate.
- **Upgrading from 0.24** chains the entries already stored and records `audit_chain_started`; the chain proves nothing about them before then.

Next steps

- Read the audit log.
- The chain and streaming, in the reference.
- Read the threat model.